

Client 1: Technical Codebook

Refreshed PSID crisis-module workflow, scoring logic, and artifact schema

Prepared for client review | Updated: April 2026 | <https://github.com/namo507/Team-PSID>

Abstract

This technical codebook documents the refreshed PSID crisis-module workflow from the ranked source bank to the scored CSV, figures, spreadsheet, and client PDFs. It explains the scoring formulas, the 28-row scored selection, the 26-question deployable instrument, and the optimization choices that now govern portability, redundancy control, and respondent-aware routing.

Keywords: PSID crisis module | utility-burden optimization | technical codebook | deployable survey design

1 Purpose and Scope

This technical codebook documents the refreshed PSID crisis-module workflow from the authoritative ranked source file through the final scored CSV, figures, spreadsheet, and PDF bundle. It is written to support analytic review, reproducibility, and downstream client sign-off.

Production status

- 52 ranked historical questions in the current corpus.
- 28 selected rows in the scored module and 26 questions in the deployable instrument.
- 3 **Generic** rows and 49 **Specific** rows after binary thresholding.
- 29.98 selected minutes in the scored module and 28.82 minutes in the deployable instrument.
- The 28-to-26 change comes from excluding **SPC_AGE_YEARS** and **SPC_DESCRIBE_SEX_GENDER** from routine field administration while retaining them for traceability.

2 Headline Benchmark Summary

Metric	Value	How the number is used
Total ranked questions	52	Count of all rows in the integrated source bank; defines the full analytic universe.
Selected scored questions	28	Count of shortlisted rows under the 30-minute review budget.
Deployable questions	26	Scored shortlist minus the two selected demographic-only traceability rows.
Selected scored minutes	29.98	Summed minutes across the 28 scored rows; confirms the review package stays inside the budget.
Deployable minutes	28.82	Summed minutes across the field instrument after age and sex/gender are removed from default administration.
Full corpus minutes	68.60	Summed minutes across all 52 rows; shows why selection is needed.
Generic rows	3	Count of rows above the normalized threshold; defines the portable universal core.
Specific rows	49	Remaining rows in the context-sensitive bank.
Average R_i	2.201	Average baseline signal-per-burden score across the refreshed corpus.
Maximum R_i	5.667	Top baseline efficiency score and normalization ceiling in the current refresh.
Average P_i	2.561	Average distinctiveness-adjusted priority score across the refreshed corpus.
Maximum P_i	7.860	Top augmented priority score after portability and redundancy adjustments.

Table 1: Current benchmark summary used by the maintained client package.

3 Why 28 Scored Rows Become 26 Deployable Questions

Stage	Rows	Minutes	Calculation / rule	Why it is retained or removed
Full ranked corpus	52	68.60	Count of all rows in the integrated source; minutes are summed across the corpus	Starting point for ranking, diagnostics, and documentation.
Scored module	28	29.98	Count of selected rows after ranking and time-budget filtering	Preserves the full analytic shortlist, including traceability fields.
Selected demographic traceability rows	2	1.17	$\text{SPC_AGE_YEARS} (0.35) + \text{SPC_DESCRIBE_SEX_GENDER} (0.817)$	Useful for framing and optional routing, but comparatively weak as direct crisis-impact questions.
Deployable instrument	26	28.82	$28 - 2 = 26$ and $29.98 - 1.167 = 28.813 \approx 28.82$	Final asked instrument prioritizes direct crisis-impact content instead of forcing demographic context into every interview.

Table 2: The change from 28 scored rows to 26 deployable questions is a deployment optimization, not a re-ranking of the corpus.

Interpretation

The model still selects 28 rows as the best analytic bundle under the budget. The deployable version removes only the two demographic-only rows because they are better treated as traceability or conditional-routing variables than as routine crisis-module questions.

4 Figures and Visual Diagnostics

Visual	Primary question answered	Why it belongs in the technical codebook
Top-ranked questions	Which items have the largest raw R_i values?	Shows the baseline efficiency leaders before portability, redundancy, and deployment filters are discussed.
Binary category comparison	How do Generic and Specific rows distribute across the contributing sources?	Verifies that the simplified binary architecture is applied across the integrated source bank rather than inside one source only.
Utility versus burden scatter	Which rows deliver the most signal at the lowest response cost?	Makes it easy to see why short, high-signal items dominate the low-burden region and survive the selection filter.
Construct heatmap	Which construct families are concentrated inside each source bank?	Explains why trauma, housing, employment, and aid items remain prominent after the refresh.
Time-budget chart	How much of the 30-minute cap is consumed by the selected Generic and Specific rows?	Connects the scoring logic to the operational deployment limit that actually determines the scored shortlist.
Function-lane diagram	Which scripts and functions own each stage of the pipeline?	Converts the implementation into an auditable production map for technical review and maintenance.
Scoring anatomy diagram	How does one question row become U_i , B_i , R_i , the binary label, and P_i ?	Shows the formula chain directly so readers do not have to infer dependencies from code alone.

Table 3: Complete visual bundle used to explain the refreshed algorithm, its diagnostics, and its delivery pipeline.

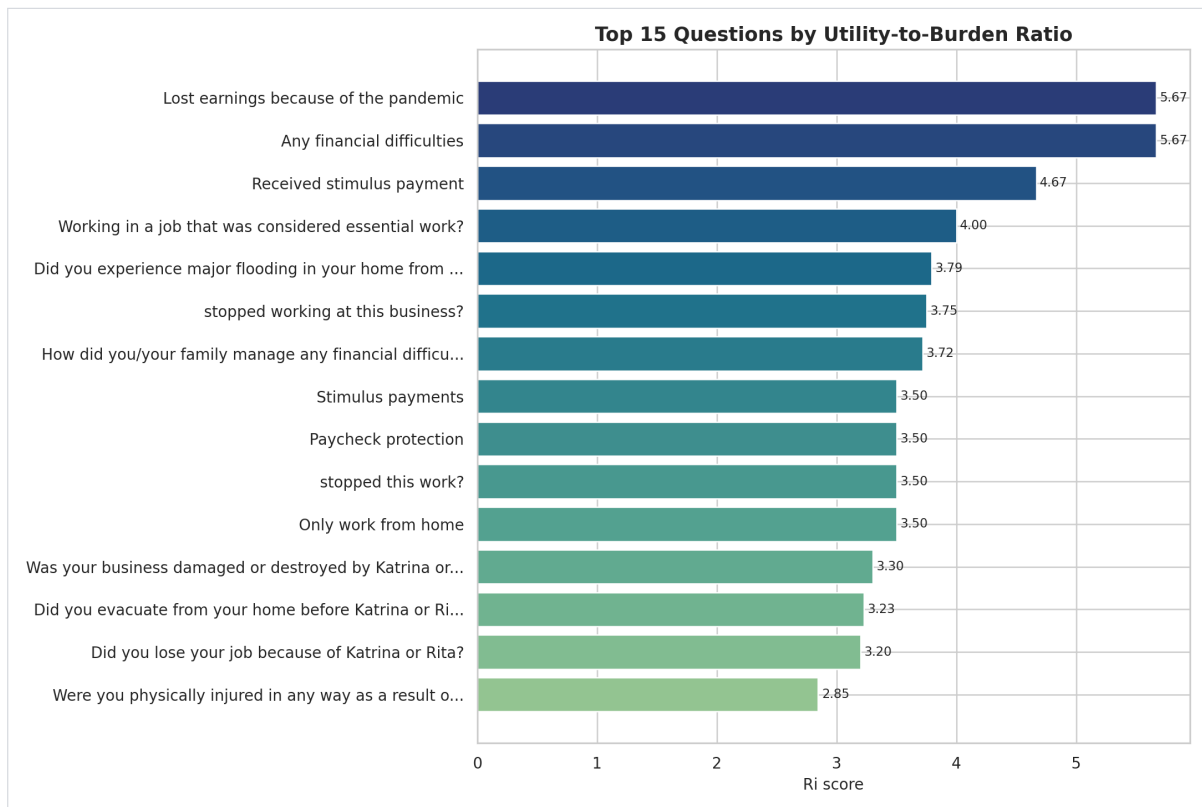


Figure 1: Top-ranked questions from the refreshed corpus.

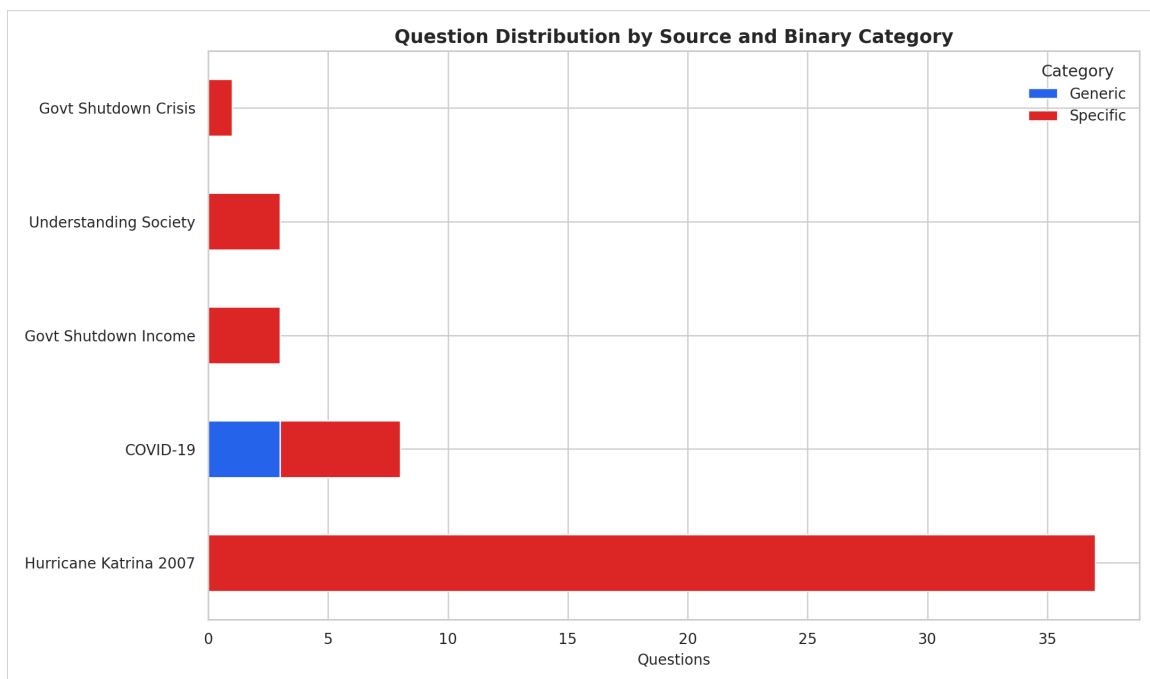


Figure 2: Binary category mix by source after the Generic versus Specific refresh.

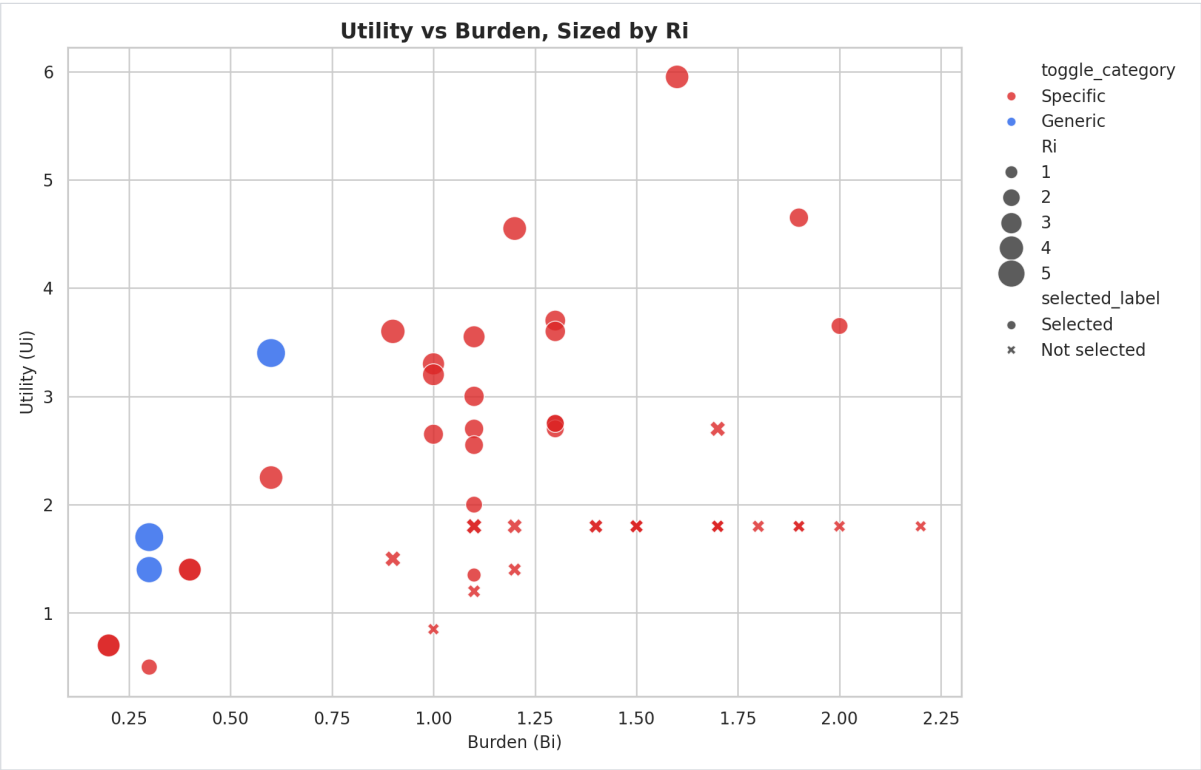


Figure 3: Utility-versus-burden view used to inspect signal efficiency and selection pressure.

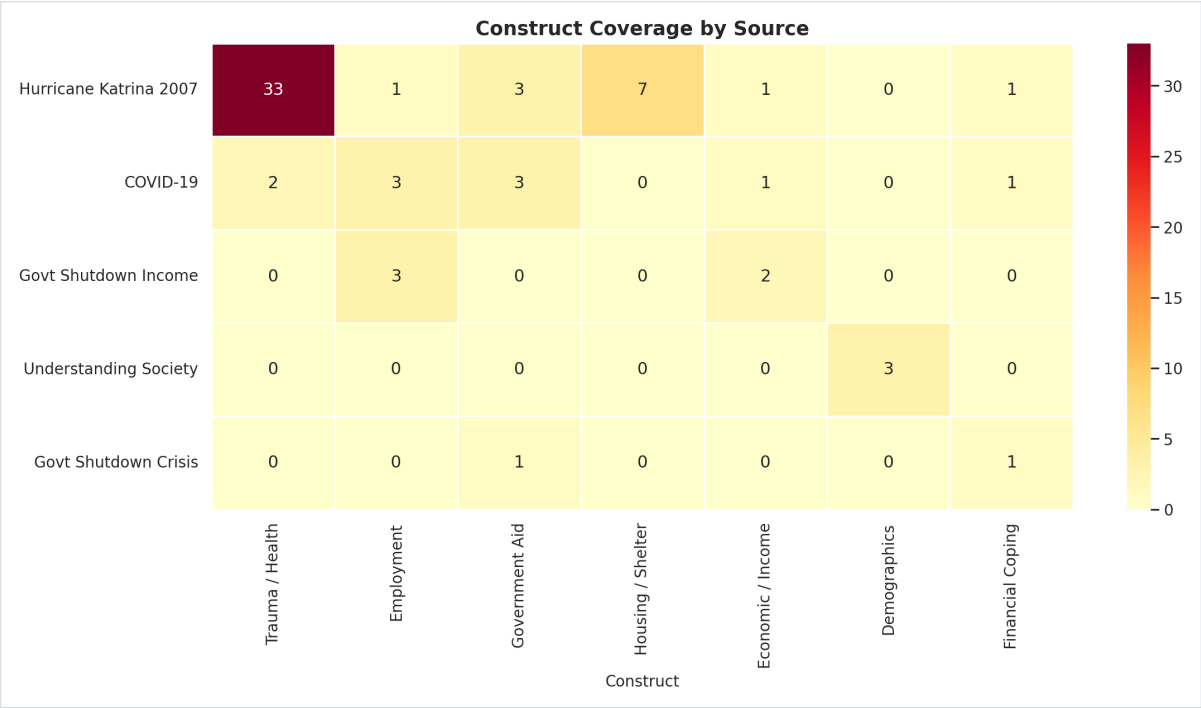


Figure 4: Construct coverage heatmap for the refreshed item bank.

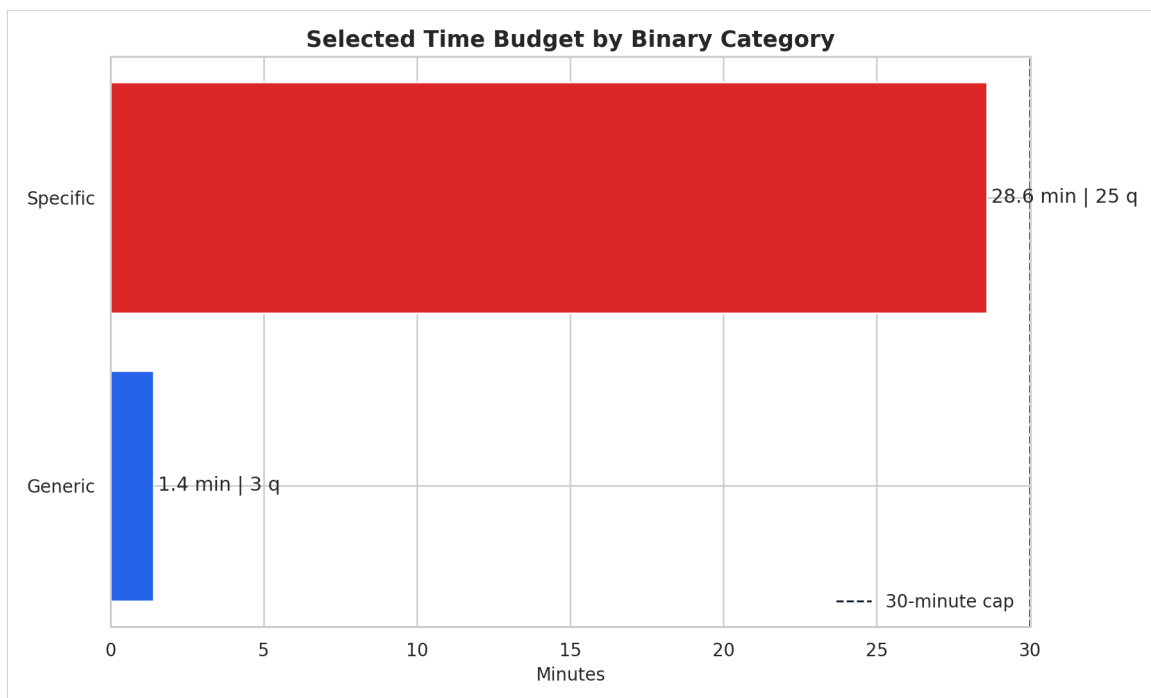


Figure 5: Selected time budget by binary category under the 30-minute cap.

5 End-to-End Workflow Pipeline

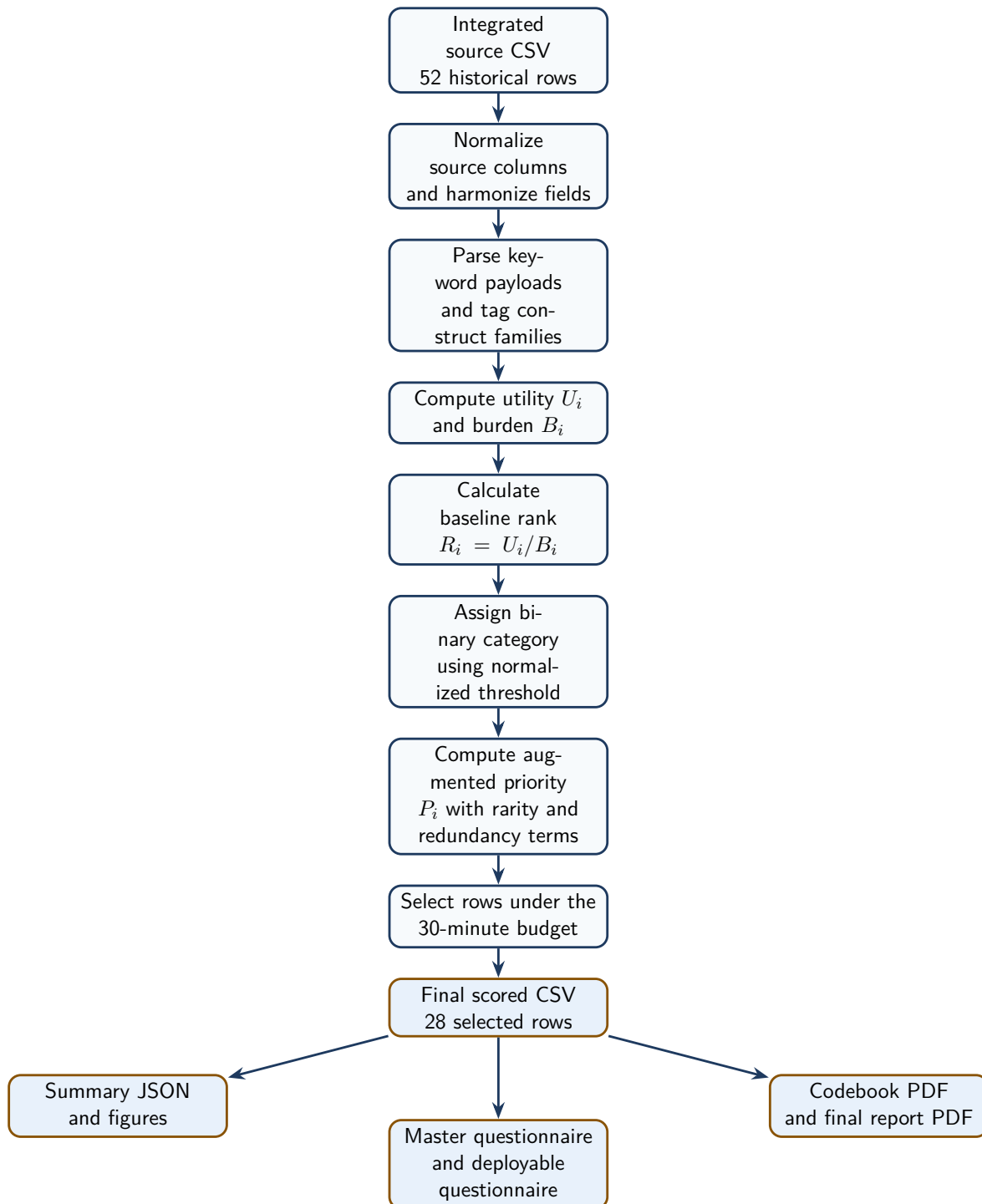


Figure 6: Production workflow from the raw integrated source to the maintained client package.

6 Function-Level Pipeline Map

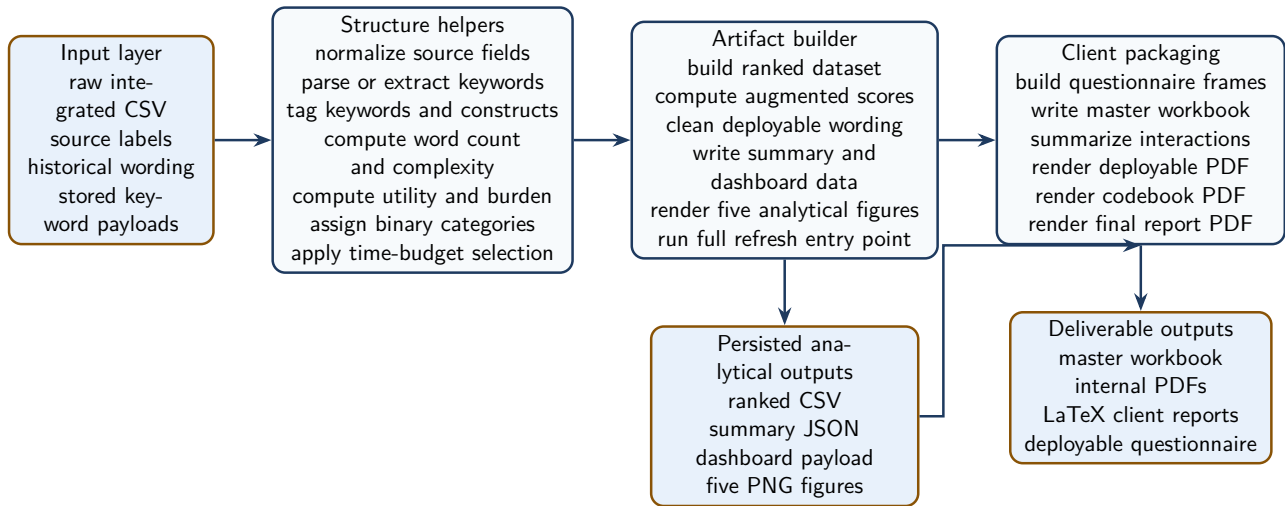


Figure 7: Function ownership across normalization, scoring, enrichment, packaging, and final delivery.

Interpretation

The implementation is intentionally split into handoff layers. Shared helpers build row-level scoring fields, the artifact script persists the ranked dataset and figures, and the packaging layer converts those same outputs into spreadsheets and PDFs. That separation is what allows one refresh path to keep the CSV, notebook, figures, and client deliverables synchronized.

7 Scoring Algorithm Anatomy

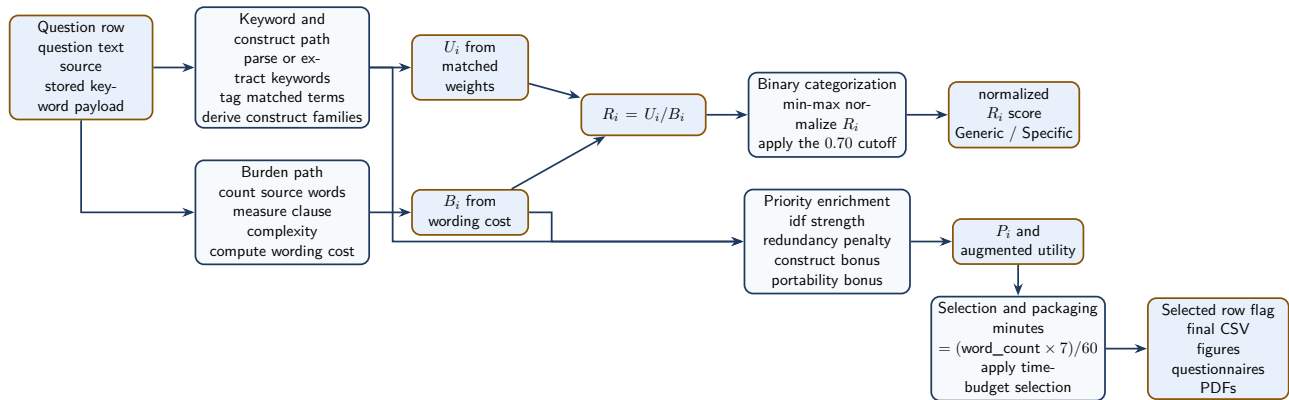


Figure 8: How one question row is transformed into baseline efficiency, a binary category label, augmented priority, and final selection.

Interpretation

The algorithm is not a single formula. It is a dependency chain. Text and stored keywords create constructs, constructs create utility, wording creates burden, R_i creates the binary label, and the enriched components create P_i . Selection happens only after both the static scores and the time budget are known.

8 Optimization Strategy Change

Dimension	Legacy architecture view	Current production view and implication
Client-facing categories	Legacy architecture exposed 7 Generic Core rows, 1 Financial Crisis row, and 20 Pandemic / Disaster rows inside the 28-row shortlist.	Current production uses 3 Generic and 25 Specific rows in the scored module, with 3 Generic and 23 crisis-specific rows in the deployable instrument. The binary story is simpler and the universal core is narrower.
Financial representation	Finance appeared as its own visible toggle label.	Financial rows now compete inside Specific and are moderated by redundancy penalties, so near-duplicate earnings and aid items no longer create a separate finance-only bank.
Deployment target	The scored shortlist was closer to the field-facing view.	The 28-row scored file is retained for auditability, while the 26-row deployable file is used for routine administration. This separates analytic traceability from operational burden.
Optimization levers	The older view emphasized score plus legacy toggle structure.	The current workflow adds portability, construct coverage, redundancy control, and documentation-backed wording replacements to support an always-on, cross-crisis instrument.
Routing refinement	Category labels described broad source-specific routing.	Generic answers and crisis identification now activate one relevant specific block, so respondent answers reduce effective burden without overwriting stored row scores.

Table 4: From the legacy multi-label architecture to the current production optimization strategy.

9 Respondent-Answer Refinement and Optimization

Refinement input	Quantitative effect	Why it matters and the assumption behind it
Universal Generic answers and crisis identification	Effective live path becomes the 3-item Generic core plus one specific block; pandemic path is $1.40 + 2.92 = 4.32$ instead of the full 26-question paper instrument.	This cuts unnecessary burden and keeps the interview aligned to the active crisis context. It assumes one dominant crisis context is active in a given administration.
Selected demographic answers	Removing SPC_AGE_YEARS (0.35) and SPC_DESCRIBE_SEX_GENDER (0.817) lowers the base deployable length by 1.167 minutes.	This preserves traceability without forcing low-yield context items into every interview. It assumes demographic information can be collected elsewhere or only when routing requires it.
Historical respondent distributions	Shutdown coping, stimulus uptake, and pandemic response patterns keep Government Aid and Financial Coping constructs salient in the selected bank.	This grounds construct weights and wording choices in observed crisis behavior rather than only lexical similarity.
Feed-forward respondent state	Prior-wave status, identity, address, and eligibility can refine universes or wording windows without overwriting stored U_i , B_i , R_i , or P_i values.	This separates static ranking from live survey routing and assumes prior-wave metadata is available when the module is deployed longitudinally.

Table 5: Respondent data influences deployment and construct emphasis even when the stored row scores remain fixed.

The respondent-data evidence used for refinement is substantive rather than cosmetic. In the reference evidence, 70.6% of shutdown-affected families cut spending, 28.3% used food banks, and pandemic response data show strong heterogeneity in vaccination and stimulus behavior. Those patterns explain why coping, aid, earnings-loss, and health-response items remain prominent after redundant variants are pruned.

10 Files and Roles

File	Role	Why it matters
data/PSID_Ranked_Questions_Katrina_Integrated.csv	Raw integrated source bank	Starting point for every production rebuild.
PSID_NLP_Crisis_Module_Structure.py	Shared scoring helpers	Holds constants, keyword parsing, burden logic, threshold helpers, and selection utilities.
generate_psid_artifacts.py	Production rebuild engine	Writes the final scored CSV, summary JSON, dashboard payload, and figures.
data/PSID_Ranked_Questions_Final.csv	Authoritative scored output	Main ranked table used by the notebook and client deliverables.
PSID_NLP_Crisis_Module_Final.ipynb	Validation notebook	Recomputes the workflow and verifies the persisted artifact bundle.
generate_client_documents.py	Client-package renderer	Builds the spreadsheet and PDFs from the scored CSV and summary JSON.
data/psid_artifact_summary.json	Metric bundle	Stores benchmark counts and score summaries for the latest run.

Table 6: Files that drive the maintained workflow and client package.

11 Core Scoring Logic

11.1 Utility, burden, and baseline rank

The baseline score is driven by question utility relative to response burden.

Core formulae

$$U_i = \sum \text{keyword weights}$$

$$B_i = \max(0.10 \times \text{word_count} + 0.20 \times \text{complexity}, 0.1)$$

$$R_i = \frac{U_i}{B_i}$$

- U_i rewards crisis-relevant content signaled by tagged keywords.
- B_i penalizes long or complex wording.
- R_i ranks items that carry more signal per unit of burden.

11.2 Binary category assignment

The client requirement was a binary rule based on R_i , but the raw score is not naturally bounded in $[0, 1]$. The production pipeline therefore applies the cutoff to a min-max normalized score.

Operational threshold rule

$$\text{ri_threshold_score} = \frac{R_i - \min(R_i)}{\max(R_i) - \min(R_i)}$$

$$\text{Generic} \iff \text{ri_threshold_score} \geq 0.70$$

$$\text{Specific} \iff \text{ri_threshold_score} < 0.70$$

11.3 Augmented priority score

The final prioritization score preserves R_i while rewarding breadth and distinctiveness.

Priority adjustment

$$\text{augmented_utility} = U_i \times \left(1 + 0.32 \times \text{idf_scaled} + 0.24 \times \text{construct_scaled} \right. \\ \left. + 0.12 \times \text{richness_scaled} + \text{portability_bonus} \right)$$

$$P_i = \frac{\text{augmented_utility}}{B_i \times (1 + 0.65 \times \text{redundancy_scaled})}$$

- P_i helps separate distinct, portable items from repetitive variants.
- The shortlist is still bounded by the time budget, not only by raw rank.

11.4 Why each quantitative field exists

Quantity	Rule	Why it matters
U_i	Sum of matched keyword weights.	Captures crisis relevance; it assumes tagged keywords are a defensible proxy for item content.
B_i	Word-count and complexity penalty with a floor at 0.1.	Encodes respondent burden; it assumes burden grows with length and cognitive complexity.
R_i	U_i/B_i .	Measures signal per unit of burden, so higher values indicate more efficient questions.
ri_threshold_score	Min-max normalization of R_i .	Makes the client's 0.70 threshold meaningful on an otherwise unbounded score.
Augmented utility	U_i adjusted by rarity, construct breadth, richness, and portability.	Rewards distinct, portable coverage before redundancy is applied.
P_i	Augmented utility divided by burden and redundancy penalty.	Breaks ties among similar R_i values by down-weighting repetitive items.
minutes	$(\text{word_count} \times 7)/60$.	Operational time field used in the budget and instrument assembly.
Selected rows	Time-budget filter after ranking.	Defines the scored module for review and traceability.
Deployable rows	Selected rows minus demographic-only traceability rows.	Defines the field instrument used in routine administration.

Table 7: Formula, purpose, and assumption guide for the main quantitative fields.

12 Worked Calculation Example

Example question: GEN_LOSE_EARNINGS_PANDEMIC

Recommended wording: *Did you lose earnings because of the pandemic?*

Field	Value	Calculation	Why it is used / assumption
U_i	3.400	$0.80 + 0.80 + 0.90 + 0.90$	Utility aggregates the matched crisis-signal weights; assumes the tagged keywords capture the substantive content of the item.
word count	6	Counted from deployable wording	Used in both burden and time estimates; assumes the cleaned wording is the administered string.
complexity	0.0	Precomputed complexity field	Prevents an additional burden penalty; assumes the wording is direct and does not require extra respondent processing.
B_i	0.600	$\max(0.10 \times 6 + 0.20 \times 0.0, 0.1)$	Converts wording cost into a penalty term; assumes the burden constants remain fixed design choices.
R_i	5.667	$3.40/0.60$	Measures signal efficiency; assumes high-value universal questions should deliver strong utility per unit of burden.
threshold score	1.000	$(5.667 - 0.818)/(5.667 - 0.818)$	Supports the binary split; assumes the current refresh min and max are the correct normalization anchors.
Final category	Generic	$1.000 \geq 0.70$	Places the item in the universal core; assumes the client wants only the top portable items to be Generic.
P_i	7.860	Value computed after rarity, construct, richness, portability, and redundancy scaling	Separates this item from similar crisis questions; assumes distinct portable items deserve priority in a short module.
minutes	0.70	$(6 \times 7)/60$	Feeds the time budget; assumes oral administration averages 7 seconds per word.

Table 8: Worked example for one of the highest-value Generic items.

13 Threshold Diagnostics

Threshold diagnostic	Value	Calculation / implication
Minimum raw R_i	0.818	$\min(R_i)$ across the 52-row corpus.
Maximum raw R_i	5.667	$\max(R_i)$ across the 52-row corpus.
Rows with raw $R_i \geq 0.70$	52 of 52	Shows that a direct raw cutoff would classify every row as Generic.
Rows with normalized threshold score ≥ 0.70	3 of 52	Final Generic count after min-max normalization.

Table 9: Why normalized thresholding is required for a meaningful binary split.

Interpretation

Raw $R_i \geq 0.70$ would classify every row as **Generic**. Normalized thresholding preserves the requested cutoff while still creating a real binary split between portable high-signal items and context-dependent items.

14 Construct Coverage and Interaction Notes

14.1 Selected construct coverage

Construct	Selected questions
Trauma / Health	15
Employment	7
Housing / Shelter	6
Government Aid	5
Economic / Income	4
Financial Coping	2
Demographics	2

Table 10: Construct coverage in the selected scored module.

14.2 Demographic interaction summary

Group	Rows	Average R_i	Average P_i
Demographics only	3	1.354	2.189
Non-demographic crisis items	49	2.253	2.584

Table 11: Demographic-only rows versus direct crisis-impact rows.

Interpretation

Demographic-only items remain useful for traceability and module framing, but they carry less direct crisis signal than employment, housing, aid, trauma, and economic disruption items. That is why the final deployable questionnaire excludes demographics even though the master review file retains them.

15 Output Schema

Field	Meaning
<code>variable_name</code>	Stable identifier used across the CSV, spreadsheet, and PDFs.
<code>question_text</code>	Historical source wording.
<code>recommended_wording</code>	Clean deployable wording.
<code>source</code>	Historical module source.
<code>toggle_category</code>	Final binary category: Generic or Specific.
<code>historical_toggle_category</code>	Legacy category kept for traceability.
<code>Ui, Bi, Ri</code>	Utility, burden, and baseline rank.
<code>ri_threshold_score</code>	Normalized rank used for binary categorization.
<code>Pi</code>	Enhanced priority score.
<code>selected_for_module</code>	Final selection flag under the time cap.
<code>minutes</code>	Estimated administration time.
<code>constructs</code>	Tagged construct families.

Table 12: Key fields preserved in the authoritative scored dataset.

16 Refresh Procedure

Use this order whenever the client package is refreshed:

1. Rebuild scored artifacts via `build_all()` in `generate_psid_artifacts.py`.

2. Validate notebook outputs in `PSID_NLP_Crisis_Module_Final.ipynb`.
3. Regenerate spreadsheet and PDFs through `generate_client_documents.py`.
4. Rebuild the client-facing LaTeX reports through `docs/latex/build_client_reports.sh`.
5. Confirm that metrics, figures, tables, and deliverable paths remain aligned.

That sequence keeps the scored CSV, the notebook, the figures, the packaged PDFs, and the presentation-ready report files synchronized.

A Function Catalog by Pipeline Stage

A.1 Input normalization and taxonomy preparation

Function / entry point	Script	Produces / updates	Why it matters
<code>normalize_ranked_questions()</code>	<code>PSID_NLP_Crisis_Module_Structure.py</code>	Harmonized column names, parsed keyword payloads, crisis-origin labels, and normalized legacy category labels	Creates the consistent schema that the downstream scoring code expects.
<code>parse_keywords()</code>	<code>PSID_NLP_Crisis_Module_Structure.py</code>	Clean Python list from CSV-stored keyword text	Lets the production CSV carry reusable keyword evidence rather than forcing extraction on every refresh.
<code>extract_keywords()</code>	<code>PSID_NLP_Crisis_Module_Structure.py</code>	RAKE phrases and noun chunks from raw question text	Backfills keyword evidence when the source row does not already include a usable keyword payload.
<code>tag_keywords()</code>	<code>PSID_NLP_Crisis_Module_Structure.py</code>	Keyword-to-construct matches with fixed weights	Converts text fragments into scored crisis constructs that can be used analytically.
<code>extract_constructs()</code>	<code>PSID_NLP_Crisis_Module_Structure.py</code>	Unique construct list attached to each row	Feeds construct coverage tables, rarity calculations, and recommendation logic.
<code>classify_toggle()</code>	<code>PSID_NLP_Crisis_Module_Structure.py</code>	Legacy multi-label interpretation of historical rows	Preserves the older architecture for traceability even though the final production split is binary.

Table 13: Functions that standardize the source bank and translate question text into construct evidence.

A.2 Baseline scoring, thresholding, and selection

Function / entry point	Script	Produces / updates	Why it matters
<code>compute_word_count()</code>	PSID_NLP_Crisis_Module_Structure.py	Source-word length for each question row	Supplies the base input for burden and time-budget calculations.
<code>compute_complexity()</code>	PSID_NLP_Crisis_Module_Structure.py	Named-entity and clause-marker burden proxy	Captures the extra respondent processing cost of longer or more complex wording.
<code>compute_utility()</code>	PSID_NLP_Crisis_Module_Structure.py	U_i from tagged keyword weights	Summarizes how much crisis signal the row carries before burden is applied.
<code>compute_burden()</code>	PSID_NLP_Crisis_Module_Structure.py	B_i from word count and complexity	Converts wording cost into the denominator used by the baseline rank.
<code>min_max_scale()</code>	PSID_NLP_Crisis_Module_Structure.py	$[0, 1]$ scaling for score vectors	Makes the client's 0.70 threshold meaningful on an otherwise unbounded raw R_i scale.
<code>assign_binary_categories()</code>	PSID_NLP_Crisis_Module_Structure.py	<code>ri_threshold_score</code> and final Generic / Specific labels	Produces the client-facing binary architecture used throughout the refreshed package.
<code>select_for_time_budget()</code>	PSID_NLP_Crisis_Module_Structure.py	Greedy shortlist under the 30-minute cap	Converts ranking into an operational module that can actually be administered.

Table 14: Functions that build the baseline rank, apply the binary cutoff, and enforce the time budget.

A.3 Augmented scoring and orchestration

Function / entry point	Script	Produces / updates	Why it matters
<code>build_ranked_dataset()</code>	<code>generate_psid_artifacts.py</code>	Fully enriched ranked dataset with derived fields	This is the core orchestration step that turns the raw integrated source into the scored analytic table.
<code>suggest_deployable_wording()</code>	<code>generate_psid_artifacts.py</code>	Cleaner crisis-portable wording proposals	Bridges historical source phrasing and field-ready client wording.
<code>compute_augmented_scores()</code>	<code>generate_psid_artifacts.py</code>	TF-IDF strength, redundancy, construct bonus, portability bonus, augmented utility, and P_i	Adds the distinctiveness layer that resolves ties among similar R_i values.
<code>build_design_recommendations()</code>	<code>generate_psid_artifacts.py</code>	Generic and Specific recommendation bundles	Converts the scored dataset into narrative design guidance rather than leaving it as a bare ranking table.
<code>build_summary()</code>	<code>generate_psid_artifacts.py</code>	Headline counts, score summaries, source mix, and construct counts	Supplies the benchmark values used throughout the notebook, dashboard, and client reports.
<code>write_dashboard_data()</code>	<code>generate_psid_artifacts.py</code>	Browser-ready dashboard payload	Keeps the interactive dashboard synchronized with the same scored dataset used everywhere else.
<code>build_all()</code>	<code>generate_psid_artifacts.py</code>	One supported refresh path for CSV, summary, dashboard data, and figures	Minimizes drift by rebuilding the full artifact bundle from one entry point.

Table 15: Functions that enrich the ranked dataset and orchestrate the maintained artifact refresh.

A.4 Figure generation and monitoring

Function / entry point	Script	Produces / updates	Why it matters
<code>plot_top_ranked()</code>	<code>generate_psid_artifacts.py</code>	Horizontal bar chart of the strongest raw R_i rows	Makes the baseline ranking visible to non-code readers.
<code>plot_toggle_comparison()</code>	<code>generate_psid_artifacts.py</code>	Source-by-category comparison chart	Shows how the binary Generic / Specific split is distributed across the contributing sources.
<code>plot_utility_vs_burden()</code>	<code>generate_psid_artifacts.py</code>	Scatter plot of U_i versus B_i , sized by R_i	Reveals the efficiency frontier that drives shortlist competition.
<code>plot_construct_heatmap()</code>	<code>generate_psid_artifacts.py</code>	Source-by-construct heatmap	Explains which crisis dimensions dominate the selected bank and why.
<code>plot_time_budget()</code>	<code>generate_psid_artifacts.py</code>	Selected-minute chart by final binary category	Connects the abstract scores to the operational 30-minute deployment constraint.

Table 16: Functions that create the complete live figure bundle used by the codebook and companion reports.

A.5 Client package assembly and final delivery

Function / entry point	Script	Produces / updates	Why it matters
<code>build_questionnaire_frames()</code>	<code>generate_client_documents.py</code>	Master and deployable questionnaire tables with universes and rounded score fields	Converts the scored CSV into the tabular structure used by review and field artifacts.
<code>write_master_questionnaire()</code>	<code>generate_client_documents.py</code>	Internal traceability workbook	Keeps the full scored selection auditable outside the notebook.
<code>build_interaction_summary()</code>	<code>generate_client_documents.py</code>	Demographic and non-demographic comparison summaries	Supplies the interaction evidence used in the final reporting layer.
<code>build_deployable_pdf()</code>	<code>generate_client_documents.py</code>	Internal deployable questionnaire PDF	Produces the field-facing question set stripped of routine demographic traceability rows.
<code>build_codebook_pdf()</code>	<code>generate_client_documents.py</code>	Internal artifact-pipeline codebook PDF	Documents the non-LaTeX packaging path used for internal review artifacts.
<code>build_report_pdf()</code>	<code>generate_client_documents.py</code>	Internal stakeholder report PDF	Gives the package a lightweight report output tied to the same scored dataset.
<code>docs/latex/build_client_reports.sh</code>	<code>docs/latex/build_client_reports.sh</code>	Final client-facing LaTeX PDFs in <code>deliverables/</code>	Compiles the maintained technical codebook, deployable questionnaire, and final report into the distribution-ready client bundle.

Table 17: Functions and entry points that turn scored artifacts into the delivered review package.

B Constants, Scaling Ranges, and Fixed Weights

B.1 Workflow constants in the live implementation

Constant	Value	Implementation rule	Why it matters
ALPHA	0.10	Adds 0.10 burden per source-word	Controls how quickly long wording reduces baseline efficiency.
BETA	0.20	Adds 0.20 burden per complexity unit	Penalizes questions that are harder to process because of entities or clause structure.
SECS_PER_WORD	7	Converts words to seconds via (word_count $\times$ 7)	Turns row length into an operational deployment cost.
MAX_SECONDS	1800	Hard cap used by the greedy selector	Enforces the 30-minute scored shortlist limit.
IDEAL_SECONDS	900	Reference lower-burden target inside the helper module	Documents the preferred shorter target even though the hard production cap remains 30 minutes.
RI_BINARY_THRESHOLD	0.70	Applied to min-max normalized R_i	Produces the final Generic / Specific split requested by the client.

Table 18: Fixed constants that determine burden, timing, and the final binary threshold.

B.2 Live scaled-component ranges from the current refresh

Input term	Minimum	Maximum	How it enters the algorithm
idf_strength	2.781	4.277	Min-max scaled to reward relatively distinctive wording inside augmented utility.
redundancy_penalty	0.000	1.000	Min-max scaled to increase the denominator of P_i for near-duplicate rows.
construct_bonus	2.146	17.863	Min-max scaled to reward rare and high-priority construct coverage.
construct_count	1	2	Min-max scaled to give extra lift to rows that cover more than one construct family.

Table 19: Current anchors used by the augmented-priority scaling logic.

B.3 Construct-priority weights used by the augmented score

Construct	Weight	Why the weight is meaningful
Trauma / Health	0.58	Highest fixed priority because direct crisis exposure, distress, and health impact are central to crisis measurement.
Housing / Shelter	0.55	Strongly weighted because displacement, damage, and housing instability are core disaster and hardship signals.
Government Aid	0.48	High enough to preserve emergency-support exposure as a major crisis-response dimension.
Financial Coping	0.45	Rewards adaptive behavior and hardship management without letting coping variants dominate the final bank.
Employment	0.42	Keeps labor-market disruption prominent but below the most direct exposure constructs.
Economic / Income	0.40	Preserves earnings and income loss as a core analytic dimension while balancing it against non-financial impacts.
Demographics	0.14	Retained for traceability and context, but intentionally weighted far below direct crisis-impact constructs.

Table 20: Fixed construct weights used inside the construct-bonus calculation.

B.4 Portability bonus rules

Row type	Source-specific term present?	Portability bonus	Why the rule exists
Generic row	No	0.16	Strongest bonus because the wording can likely survive across multiple crisis contexts without rewriting.
Generic row	Yes	0.03	Still rewards high-value generic items, but dampens the bonus when the row carries event-specific wording.
Specific row	No	0.08	Rewards reusable specific items that still need context-sensitive routing.
Specific row	Yes	0.02	Smallest bonus because these rows are useful but clearly tied to one historical crisis or program.

Table 21: Portability bonus rules used inside the augmented-priority calculation.

C Detailed Formula Derivations and Current Value Explanations

C.1 Base scoring pipeline

For question i , the code builds the numeric fields in the following order:

$$\begin{aligned}
\text{keyword_list}_i &= \text{parse_keywords}(\text{keywords}_i) \text{ or } \text{extract_keywords}(\text{question_text}_i) \\
\text{tagged_keywords}_i &= \text{tag_keywords}(\text{keyword_list}_i) \\
U_i &= \sum_{k \in \text{tagged_keywords}_i} w_k \\
\text{word_count}_i &= \text{len}(\text{split}(\text{question_text}_i)) \\
\text{complexity}_i &= \# \text{entities}_i + 0.5(\# \text{commas}_i + \# \text{semicolons}_i + \# \text{"or"}_i + \# \text{"and"}_i) \\
B_i &= \max(0.10 \times \text{word_count}_i + 0.20 \times \text{complexity}_i, 0.1) \\
R_i &= \frac{U_i}{B_i}
\end{aligned}$$

How to interpret these values

- U_i increases when more high-weight crisis constructs are matched.
- **word_count** is taken directly from the stored source wording and feeds both burden and time estimates.
- **complexity** increases when a question contains more named entities or clause separators, which acts as a proxy for respondent processing burden.
- B_i has a floor of 0.1 so that very short questions never create an undefined or explosively large ratio.
- R_i is the baseline efficiency score: more crisis signal per unit of burden yields a larger value.

C.2 Scaled component formulae for augmented scoring

Scaled field	Formula	Current anchor values	Why it matters
<code>idf_scaled</code>	$(idf_strength - 2.781)/(4.277 - 2.781)$	Minimum 2.781, maximum 4.277	Rewards comparatively distinctive wording patterns in the current refresh.
<code>redundancy_scaled</code>	$(redundancy_penalty - 0.000)/(1.000 - 0.000)$	Minimum 0.000, maximum 1.000	Increases the denominator of P_i when a row is very similar to another row in the bank.
<code>construct_scaled</code>	$(construct_bonus - 2.146)/(17.863 - 2.146)$	Minimum 2.146, maximum 17.863	Rewards rows that cover rarer or more important constructs.
<code>richness_scaled</code>	$(construct_count - 1)/(2 - 1)$	Minimum 1, maximum 2	Gives extra lift to rows that span more than one construct family.

Table 22: Current scaling rules applied before the augmented-priority score is calculated.

C.3 Binary thresholding, time-budget selection, and deployable minutes

The final client-facing category model uses a normalized version of R_i :

$$ri_threshold_score_i = \frac{R_i - \min(R)}{\max(R) - \min(R)}$$

Generic $\iff ri_threshold_score_i \geq 0.70$
 Specific $\iff ri_threshold_score_i < 0.70$

The scored shortlist is then chosen by a greedy time-budget filter:

$$question_seconds_i = word_count_i \times 7$$

$$selected_for_module_i = 1 \text{ if adding row } i \text{ keeps cumulative time } \leq 1800 \text{ seconds}$$

The deployable instrument length is then derived from the scored shortlist, not from a second ranking pass:

$$\begin{aligned} \text{deployable minutes} &= 29.98 - 0.35 - 0.817 \\ &= 28.813 \approx 28.82 \end{aligned}$$

The subtraction removes `SPC_AGE_YEARS` and `SPC_DESCRIBE_SEX_GENDER` from routine field

administration while leaving them available in the traceability record.

C.4 Augmented priority score and why values change after enrichment

The production workflow does not stop at R_i . It also computes a distinctiveness-adjusted priority score that favors portable and non-redundant items.

$$\begin{aligned} \text{idf_strength}_i &= \text{mean}(\text{TF-IDF weights of analyzed tokens}) \\ \text{redundancy_penalty}_i &= \max_j \text{cosine_similarity}(i, j) \\ \text{rarity}_i &= \text{mean}\left(\frac{1}{\text{construct frequency}}\right) \times N \\ \text{priority}_i &= \text{mean}(\text{construct weights}) \\ \text{construct_bonus}_i &= \text{rarity}_i + \text{priority}_i + 0.08 \times \text{construct_count}_i \end{aligned}$$

All scaled components use min-max scaling within the current refreshed dataset, and the final augmented score is:

$$\begin{aligned} \text{augmented_utility}_i &= U_i \times \left(1 + 0.32 \text{idf_scaled}_i + 0.24 \text{construct_scaled}_i \right. \\ &\quad \left. + 0.12 \text{richness_scaled}_i + \text{portability_bonus}_i\right) \\ P_i &= \frac{\text{augmented_utility}_i}{B_i \times \left(1 + 0.65 \text{redundancy_scaled}_i\right)} \end{aligned}$$

Why P_i can differ from R_i

Two questions can have similar baseline efficiency but very different P_i values. A question moves up when it adds rare or high-priority constructs, uses portable wording, and is not a near-duplicate of other shortlisted items. A question moves down when it is redundant, overly source-specific, or substantively repetitive even if its raw R_i is acceptable.

D Worked Examples From the Current Scored Dataset

D.1 Example A: Generic high-signal earnings item

Variable: GEN_LOSE_EARNINGS_PANDEMIC

Recommended wording: *Did you lose earnings because of the pandemic?*

Constructs: Economic / Income and Trauma / Health.

Field	Value	Calculation / rule	Why the value matters
U_i	3.400	$0.80 + 0.80 + 0.90 + 0.90$ from the matched keywords <i>earnings</i> , <i>lost earnings</i> , <i>pandemic</i> , and <i>the pandemic</i>	The row carries strong signal from both income shock and crisis-context wording.
B_i	0.600	$\max(0.10 \times 6 + 0.20 \times 0.0, 0.1)$	Very low wording cost keeps the denominator small.
R_i	5.667	$3.40/0.60$	Reaches the raw ceiling of the current refreshed corpus.
idf_scaled	0.946	$(4.196 - 2.781)/(4.277 - 2.781)$	The wording is distinctive relative to the rest of the bank.
construct_bonus	7.893	$7.243 + 0.490 + 0.160$	The row benefits from rarity, high-priority constructs, and multi-construct coverage.
construct_scaled	0.366	$(7.893 - 2.146)/(17.863 - 2.146)$	Keeps the construct reward meaningful without letting it dominate the score.
redundancy_scaled	0.170	$(0.170 - 0.000)/(1.000 - 0.000)$	Only a mild penalty is applied for similarity to other rows.
portability_bonus	0.03	Generic row with a source-specific term present	The row remains valuable enough to stay Generic even though the wording still names a crisis context.
augmented_utility	5.237	$3.40 \times (1 + 0.32 \times 0.946 + 0.24 \times 0.366 + 0.12 \times 1.000 + 0.03)$	Distinctiveness and construct coverage lift the baseline utility substantially.
P_i	7.860	$5.237/[0.60 \times (1 + 0.65 \times 0.170)]$	This is the strongest augmented-priority score in the current refresh.
minutes	0.70	$(6 \times 7)/60$	The row is analytically strong and operationally cheap.

Table 23: Worked derivation for the highest-priority Generic row in the current scored dataset.

D.2 Example B: Specific employment-exposure item

Variable: SPC_WORKING_JOB_CONSIDERED_ESSENTIAL

Recommended wording: *Were you working in a job that was considered essential during the crisis?*

Construct: Employment.

Field	Value	Calculation / rule	Why the value matters
U_i	3.600	$0.70 + 0.75 + 0.75 + 0.70 + 0.70$ from <i>a job, considered essential work,</i> <i>essential work, job, and working</i>	Dense employment signal keeps the item competitive even without multiple constructs.
B_i	0.900	$\max(0.10 \times 9 + 0.20 \times 0.0, 0.1)$	The wording is longer than the Generic screeners but still moderate in burden.
R_i	4.000	$3.60/0.90$	Strong baseline efficiency keeps the row near the top of the Specific bank.
idf_scaled	0.867	$(4.078 - 2.781)/(4.277 - 2.781)$	The wording remains relatively distinctive within the corpus.
construct_bonus	7.929	$7.429 + 0.420 + 0.080$	Rarity does most of the work because the row covers only one construct family.
construct_scaled	0.368	$(7.929 - 2.146)/(17.863 - 2.146)$	Provides a moderate construct lift without a richness bonus.
redundancy_scaled	0.165	$(0.165 - 0.000)/(1.000 - 0.000)$	Similarity pressure is low, so the denominator penalty stays modest.
portability_bonus	0.08	Specific row with no source-specific term present	The wording is reusable across crises even though it remains a routed specific item.
augmented_utility	5.205	$3.60 \times (1 + 0.32 \times 0.867 + 0.24 \times 0.368 + 0.12 \times 0.000 + 0.08)$	Portability and distinctiveness lift the row above a simple single-construct baseline.
P_i	5.223	$5.205/[0.90 \times (1 + 0.65 \times 0.165)]$	Highest-priority Specific item in the current refresh because it balances clarity, portability, and low redundancy.
minutes	1.05	$(9 \times 7)/60$	The row remains deployable despite being more detailed than the Generic core.

Table 24: Worked derivation for the strongest current Specific employment item.

D.3 Example C: Long-form coping item with a redundancy penalty

Variable: SPC_HOUSEHOLD_MANAGE_FINANCIAL_DIFFICULT

Recommended wording: *How did your household manage financial difficulties caused by the shutdown or crisis?*

Constructs: Financial Coping and Government Aid.

Field	Value	Calculation / rule	Why the value matters
U_i	5.950	$0.85 + 0.75 + 0.70 + 0.85 + 0.75 + 0.65 + 0.65 + 0.75$ from the matched coping and shutdown keywords	Substantively rich rows can accumulate very high utility before burden is applied.
B_i	1.600	$\max(0.10 \times 16 + 0.20 \times 0.0, 0.1)$	The longer source wording sharply increases the denominator.
R_i	3.719	$5.95/1.60$	Strong but no longer dominant once the high burden is counted.
idf_scaled	0.826	$(4.017 - 2.781)/(4.277 - 2.781)$	Distinctive wording still helps the row recover some rank after the burden penalty.
construct_bonus	13.006	$12.381 + 0.465 + 0.160$	The row covers rare and policy-relevant constructs, so the construct bonus is large.
construct_scaled	0.691	$(13.006 - 2.146)/(17.863 - 2.146)$	Strong construct coverage provides meaningful uplift in the augmented score.
redundancy_scaled	0.418	$(0.418 - 0.000)/(1.000 - 0.000)$	Similar coping variants in the bank create a much larger denominator penalty.
portability_bonus	0.02	Specific row with a source-specific term present	The row is useful but clearly tied to one historical context, so the portability reward is small.
augmented_utility	9.343	$5.95 \times (1 + 0.32 \times 0.826 + 0.24 \times 0.691 + 0.12 \times 1.000 + 0.02)$	Augmented utility remains very high because the row is analytically rich.
P_i	4.592	$9.343/[1.60 \times (1 + 0.65 \times 0.418)]$	The final priority drops below shorter items because burden and redundancy dominate the denominator.
minutes	1.87	$(16 \times 7)/60$	This row shows exactly why the final algorithm cannot rely on raw utility alone.

Table 25: Worked derivation for a high-utility row that is pushed down by length and redundancy.

Cross-example takeaway

The three live examples show the actual tradeoffs enforced by the production algorithm. High signal and low burden lift R_i , distinctiveness and construct breadth lift P_i , and long or repetitive wording suppresses otherwise strong candidates. That is the logic the figures, tables, and final 28-row selection are meant to make visible.